Lecture 4: Classical Logic and Quantifiers

Stefano M. Nicoletti

1 Lecture 4: Classical Logic and Quantifiers

1.1 Aim of the lecture

Lecture 3 connected Lean tactics with the introduction and elimination rules of natural deduction. This lecture extends that framework in two directions. We first introduce reductio ad absurdum as a classical rule. We then move from propositional logic to predicate logic and learn to represent objects, properties, functions, relations, and quantifiers.

The lecture will:

- distinguish constructive rules from classical principles;
- apply reductio ad absurdum;
- build a small first-order language in Lean;
- formalize statements containing `∀` and `∃`;
- identify and apply `∀I`, `∀E`, `∃I`, and `∃E`.

1.2 From constructive rules to classical logic

The rules seen so far have a direct constructive reading: each proof specifies how to build the required term. To prove a conjunction we construct both components; to prove an implication we construct a function; to prove an existential statement, as we will see, we provide a witness.

In Lean's constructive base logic, excluded middle `A ∨ ¬A` and double-negation elimination `¬¬A → A` are not automatically available. Lean does allow classical principles to be used locally and explicitly (Source: Theorem Proving in Lean 4, Classical Logic).

1.3 Reductio ad absurdum

Classical reductio ad absurdum proves `A` by showing that assuming `¬A` leads to `False`. In Lean, we apply it with `Classical.byContradiction`.

$$\dfrac{\dfrac{}{\neg A}\ 1 \quad \begin{matrix}\vdots \\ \bot\end{matrix}}{A}\ 1\ \ \text{RAA}$$

Reductio ad absurdum, RAA

The working pattern is: apply reductio, introduce `¬A`, derive `False` using the available assumptions, and use exactly that contradiction.

```
theorem lecture04_reductio_ad_absurdum
    (Rains TakeUmbrella : Prop)
    (hNotRainsNotUmbrella : ¬Rains → ¬TakeUmbrella)
    (hTakeUmbrella : TakeUmbrella) :
    Rains := by
  apply Classical.byContradiction
  intro hNotRains
  have hNotTakeUmbrella := hNotRainsNotUmbrella hNotRains
  have hContradiction := hNotTakeUmbrella hTakeUmbrella
  exact hContradiction
```

The first assumption turns `¬Rains` into `¬TakeUmbrella`. The latter is a function from `TakeUmbrella` to `False`; applying it to `hTakeUmbrella` produces the required contradiction.

1.4 A limitation of propositional logic

In propositional logic, each complete statement is represented by a separate proposition. We might use `P` for "Hypatia is a philosopher," `Q` for "Hannah Arendt is a philosopher," and `R` for "Hypatia is curious." This hides the fact that `P` and `Q` attribute the same property to different objects. It also prevents us from directly expressing "every philosopher is curious" or "there is a curious philosopher."

Predicate logic exposes the objects we discuss and the properties or relations attributed to them.

1.5 The domain of discourse

Every formalization begins with a *domain*: the collection of objects under consideration. In this lecture we use one general type.

```
variable (Thing : Type)
```

`Thing` represents the domain of discourse. A term `x : Thing` is an element of that domain. This is the usual single-domain presentation of first-order logic (Source: Logic and Proof, First Order Logic in Lean).

Named objects are represented by terms of the domain:

```
variable (Thing : Type)
variable (hypatia hannah rome athens : Thing)
```

These declarations do not yet assert any proposition. They only introduce terms that will receive an interpretation.

1.6 Predicates: properties of objects

A one-place predicate takes an object and returns a proposition.

```
variable (Thing : Type)
variable (Person City Philosopher Curious : Thing → Prop)
variable (hypatia hannah : Thing)

#check Philosopher hypatia
#check Philosopher hannah
```

`Philosopher hypatia` and `Philosopher hannah` are different propositions, but they share the same predicate.

Natural language	Lean
Hypatia is curious	`Curious hypatia`
Hannah Arendt is curious	`Curious hannah`
Io is a moon of Jupiter	`MoonOfJupiter io`
It rains in Rome	`RainsIn rome`

1.7 Functions: constructing new terms

A function takes one or more objects from the domain and returns an object.

```
variable (Thing : Type)
variable (hypatia : Thing)
variable (motherOf : Thing → Thing)
variable (Curious : Thing → Prop)

#check motherOf hypatia
#check Curious (motherOf hypatia)
```

`motherOf hypatia : Thing` is a new term. Since its type is still `Thing`, it can be used as the argument of a predicate. Functions can be nested: `motherOf (motherOf hypatia)` denotes the mother of Hypatia's mother.

In first-order logic a function is total and returns exactly one result for each argument. A function returns an object; a predicate returns a proposition.

1.8 Relations between objects

A predicate with two or more arguments expresses a relation.

```
variable (Thing : Type)
variable (hypatia hannah athens : Thing)
variable (Knows Visits : Thing → Thing → Prop)

#check Knows hypatia hannah
#check Visits hypatia athens
```

Argument order matters. `Knows hypatia hannah` and `Knows hannah hypatia` represent different statements.

1.9 Local parameters and assumptions

The keyword `variable` introduces local parameters. A `section` limits their scope, and Lean adds to a theorem only the parameters actually used in its statement.

```
section PredicateLogicLanguage

variable (Thing : Type)
variable (hypatia hannah : Thing)
variable (Philosopher Curious : Thing → Prop)
variable (motherOf : Thing → Thing)
variable (Knows : Thing → Thing → Prop)

end PredicateLogicLanguage
```

Declaring `Philosopher : Thing → Prop` does not assert that anyone is a philosopher. A genuine assumption that Hypatia is a philosopher has the form `hHypatiaPhilosopher : Philosopher hypatia`. The vocabulary determines which propositions can be formulated; assumptions provide proofs of some of them.

1.10 Variables and quantifiers

A variable `x : Thing` represents an object that has not been specified. Quantifiers turn expressions containing such variables into statements:

- ∀ reads "for every";
- ∃ reads "there exists at least one."

Natural language	Lean
Every person is curious	`∀ x : Thing, Person x → Curious x`
Some person is curious	`∃ x : Thing, Person x ∧ Curious x`
Every philosopher is curious	`∀ x : Thing, Philosopher x → Curious x`
There is a curious philosopher	`∃ x : Thing, Philosopher x ∧ Curious x`

Writing `x : Thing` explicitly makes both the introduced variable and its domain visible.

1.11 Implication and conjunction under quantifiers

Two patterns occur repeatedly:

```
variable (Thing : Type)
variable (Philosopher Curious : Thing → Prop)

#check ∀ x : Thing, Philosopher x → Curious x
#check ∃ x : Thing, Philosopher x ∧ Curious x
```

In the universal formula we take an arbitrary thing and say: *if* it is a philosopher, then it is curious. The implication restricts the assertion to philosophers. In the existential formula we seek one witness that is *both* a philosopher *and* curious, so the properties are joined by a conjunction.

```
variable (Thing : Type)
variable (Student Reads Understands : Thing → Prop)

#check ∀ x : Thing, Student x → Reads x
#check ∃ x : Thing, Student x ∧ Reads x
#check ∀ x : Thing, Student x → Reads x → Understands x
#check ∃ x : Thing, Student x ∧ Reads x ∧ Understands x
```

`#check` verifies that an expression is a well-formed proposition. It does not prove that proposition.

1.12 Functions inside formulas

Terms built with functions can occur inside predicates, relations, and quantified statements.

```
variable (Thing : Type)
variable (hypatia : Thing)
variable (Student Reads Curious : Thing → Prop)
variable (motherOf : Thing → Thing)
variable (Knows : Thing → Thing → Prop)

#check Curious (motherOf hypatia)
#check ∀ x : Thing, Student x → Knows x (motherOf x)
#check ∃ x : Thing, Student x ∧ Reads (motherOf x)
```

The parentheses show that we first build `motherOf x` and then use that term as the argument of a predicate or relation.

1.13 Nested quantifiers

Quantifier order determines which choices may depend on others.

```lean
variable (Thing : Type)
variable (Person : Thing → Prop)
variable (Knows : Thing → Thing → Prop)

-- Every person knows someone.
#check ∀ x : Thing, Person x →
  ∃ y : Thing, Person y ∧ Knows x y

-- Someone knows every person.
#check ∃ x : Thing, Person x ∧
  ∀ y : Thing, Person y → Knows x y

-- Nobody knows every person.
#check ∃¬ x : Thing, Person x ∧
  ∀ y : Thing, Person y → Knows x y
```

In the first statement, the witness `y` may change when `x` changes. The second statement requires one fixed `x` who knows every relevant `y`.

1.14 Universal introduction, '∀I'

$$\frac{A(x)}{\forall y\, A(y)}\ \forall\mathrm{I}$$

Universal quantifier introduction, ∀I

To prove `∀ y : Thing, A y`, introduce an arbitrary `x`. The goal becomes `A x`. Prove it without using any special property of `x`; the conclusion then holds for every object in the domain.

```lean
theorem lecture04_forall_intro
    (Thing : Type)
    (Person Curious : Thing → Prop) :
    ∀ y : Thing, Person y → Curious y → Curious y := by
  intro x
  intro hXPerson
  intro hXCurious
  exact hXCurious
```

Only the first `intro` applies `∀I`. The following two introduce the assumptions of the implications.

1.14.1 The arbitrariness condition

The object introduced by `∀I` cannot be a particular object selected because of a special property. In Lean, `intro x` introduces a fresh object of the type over which we are quantifying.

1.15 Universal elimination, ‘∀E‘

$$\frac{\forall x\, A(x)}{A(t)}\ \forall\mathrm{E}$$

Universal quantifier elimination, ∀E

Given `∀ x : Thing, A x`, choose a term `t` and apply the universal proof to it. This yields `A t`.

```
theorem lecture04_forall_elim
    (Thing : Type)
    (Student Reads : Thing → Prop)
    (marta : Thing)
    (hEveryStudentReads : ∀ x : Thing, Student x → Reads x)
    (hMartaStudent : Student marta) :
    Reads marta := by
  have hIfMartaStudentThenReads := hEveryStudentReads marta
  have hMartaReads := hIfMartaStudentThenReads hMartaStudent
  exact hMartaReads
```

Applying the universal assumption to `marta` produces `Student marta → Reads marta`, which can then be applied to `hMartaStudent`.

1.16 Existential introduction, ‘∃I‘

$$\frac{A(t)}{\exists x\, A(x)}\ \exists\mathrm{I}$$

Existential quantifier introduction, ∃I

To prove `∃ x : Thing, A x`, choose a witness `t` and prove `A t`. Lean’s constructor is `Exists.intro`.

```
theorem lecture04_exists_intro
    (Thing : Type)
    (Reads : Thing → Prop)
    (marta : Thing)
    (hMartaReads : Reads marta) :
    ∃ x : Thing, Reads x := by
  apply Exists.intro marta
  exact hMartaReads
```

Choosing `marta` turns the existential goal into `Reads marta`, which is already available as an assumption.

1.17 Existential elimination, '∃E'

$$\dfrac{\exists x\, A(x) \qquad \begin{array}{c}\overline{A(y)}^{\,1}\\ \vdots \\ B\end{array}}{B}\ \exists\mathrm{E}^{1}$$

Existential quantifier elimination, ∃E

Given `∃ x : Thing, A x`, we know that a witness exists but may not assume its identity. To derive `B`, introduce an arbitrary witness `y` together with `A y`, and derive a conclusion that does not depend on the identity of `y`.

```
theorem lecture04_exists_elim
    (Thing : Type)
    (FiledReport : Thing → Prop)
    (InvestigationStarts : Prop)
    (hSomeoneFiledReport : ∃ x : Thing, FiledReport x)
    (hReportStartsInvestigation :
      ∀ x : Thing, FiledReport x → InvestigationStarts) :
    InvestigationStarts := by
  apply Exists.elim hSomeoneFiledReport
  intro y
  intro hYFiledReport
  have hInvestigation :=
    hReportStartsInvestigation y hYFiledReport
  exact hInvestigation
```

The conclusion `InvestigationStarts` contains no occurrence of `y` and does not depend on the witness's identity.

1.18 Combining the rules

The quantifier rules are often used together.

```
theorem lecture04_exists_elim_combined
    (Thing : Type)
    (HasUmbrella StaysDry : Thing → Prop)
    (hSomeoneHasUmbrella : ∃ x : Thing, HasUmbrella x)
    (hUmbrellaStaysDry : ∀ x : Thing, HasUmbrella x → StaysDry x) :
    ∃ x : Thing, StaysDry x := by
  apply Exists.elim hSomeoneHasUmbrella
  intro x
  intro hXHasUmbrella
  apply Exists.intro x
  have hXStaysDry := hUmbrellaStaysDry x hXHasUmbrella
  exact hXStaysDry
```

Here `Exists.elim` opens the first existential, applying `hUmbrellaStaysDry` to `x` uses ∀E, and `Exists.intro x` uses the same object as the witness of the new existential.

1.19 Working strategy

When reading a quantified formula, ask:

- Does the goal begin with ∀? Introduce an arbitrary object with `intro`.
- Is there a universal assumption? Apply it to a term from the domain.
- Does the goal begin with ∃? Choose a witness with `Exists.intro`.
- Is there an existential assumption? Open it with `Exists.elim` without presupposing the witness's identity.
- Are several quantifiers nested? Check their order and dependencies.

This extends the strategy used for propositional connectives: inspect the main constructor of the goal for an introduction rule and inspect compound assumptions for useful elimination rules.

1.20 Sources and further reading

- (Source: Jeremy Avigad, Robert Y. Lewis, and Floris van Doorn, Logic and Proof: Natural Deduction Rules).
- (Source: The Lean Language Reference, Quantifiers).
- (Source: Logic and Proof, First Order Logic in Lean).
- (Source: Theorem Proving in Lean 4, Classical Logic).